Cross-Platform Performance Evaluation of Serverless Computing A Comparative Study of AWS Lambda and Azure Functions

Anna Makridou
csdp1448@csd.uoc.gr
University of Crete, Computer Science Department
Heraklion, Greece

Abstract

Serverless computing platforms like AWS Lambda and Azure Functions have gained widespread adoption, yet lack comprehensive performance comparisons using identical code and fair experimental conditions. This paper presents a rigorous microbenchmark study comparing AWS Lambda and Azure Functions across seven diverse workloads: four function benchmarks (HTML generation, image processing, file compression, and a custom matrix multiplication benchmark) and three workflow orchestration patterns (sequential, parallel, and map-reduce). Using the SeBS-Flow framework with identical code and inputs, 135+ invocations were executed across both platforms, achieving 100% success rate. Results show Azure wins all function performance categories (4/4 workloads, 8/8 metrics), with 4.6–22.6× faster cold starts and up to 4.9× faster warm execution for production workloads. For workflows, AWS demonstrates consistent advantages across all three patterns: 10.6× faster for sequential workflows, 20.8× faster for parallel execution, and 15.3× faster for complex map-reduce.

CCS Concepts

• **CS-559 Project** → **Cloud computing**; • **Networks** → *Network performance evaluation.*

Keywords

Serverless computing, FaaS, AWS Lambda, Azure Functions, Benchmarking, Performance evaluation, Cold starts, Warm Starts, Experiments, Platform Comparison

ACM Reference Format:
Anna Makridou. 2026. Cross-Platform Performance Evaluation of Serverless Computing A Comparative Study of AWS Lambda and Azure Functions. In *Proceedings of CS-559 Project*. ACM, New York, NY, USA, 5 pages. https://doi.org/559.559

1 Introduction

Serverless computing has revolutionized application deployment by abstracting infrastructure management and enabling automatic scaling.

Major cloud providers including Amazon Web Services (AWS Lambda), Microsoft Azure (Azure Functions) and Google Cloud Platform (GCP Cloud Functions) offer serverless platforms with distinct architectures, pricing models, and performance characteristics.

Despite widespread adoption, developers face significant challenges in selecting appropriate platforms due to limited comparative performance data. Existing studies often use platform-specific code, different input sizes or focus on single workload types, making fair comparisons difficult. Additionally, most research examines either function-level performance or workflow orchestration, but not both systematically.

This work focuses on AWS and Azure because they were the only platforms available through our educational accounts (AWS Academy and Azure for Students). These programs provided consistent credits and credential lifetimes needed to run 135+ invocations with identical code.

1.1 Motivation

This study addresses three critical gaps in serverless performance evaluation:

(1) **Fair Comparison**: Most benchmarks use platform-specific implementations, making it unclear whether performance differences stem from platform capabilities or code optimization.
(2) **Workload Diversity**: Existing studies typically focus on single workload types, missing insights into how platforms perform across diverse application patterns.

1.2 Objectives

The research objectives are:

(1) **Validate Framework**: Confirm that SeBS-Flow, an open-source serverless benchmarking suite, works on current AWS and Azure infrastructure.
(2) **Fair Performance Comparison**: Compare AWS Lambda and Azure Functions using identical code, inputs and experimental methodology across diverse workloads.
(3) **Workload Characterization**: Analyze performance across four distinct workload types: pure compute, I/O-intensive, CPU-intensive and lightweight operations.
(4) **Workflow Evaluation**: Compare AWS Step Functions and Azure Durable Functions for serverless workflow orchestration.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CS-559 Project, Heraklion, Crete
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-000-0/2026/01
https://doi.org/559.559

2 Background

2.1 Serverless Computing

Serverless computing abstracts infrastructure management, enabling developers to deploy code without managing servers. Key characteristics include:

- **Event-driven execution**: Functions triggered by HTTP requests, database changes and file uploads.
- **Automatic scaling**: Platform scales from zero to thousands of concurrent executions.
- **Pay-per-use pricing**: Charges based on execution time and memory, not idle resources.
- **Cold starts**: Initial invocation latency due to container initialization.

2.2 Platform Architectures

AWS Lambda uses a library-based trigger mechanism with direct function invocation through the AWS SDK. Functions execute in Firecracker microVMs with initialization performed once per container lifecycle.

Azure Functions employs an HTTP-based trigger mechanism where all invocations route through HTTP endpoints. This architectural difference introduces network overhead but provides platform-agnostic invocation methods.

3 Experiments

3.1 Experimental Setup

3.1.1 Infrastructure Configuration. Table 1 shows the experimental configuration. AWS us-east-1 was used for AWS Lambda. Azure francecentral was selected for Azure Functions due to Azure for Students account restrictions, limiting deployment to five specific EU regions (francecentral, polandcentral, norwayeast, switzerlandnorth, germanywes-central), preventing deployment to US regions. Both platforms ran Python 3.8 runtimes with 128–256 MB memory allocations, matching typical microservice configurations.

Table 1: Experimental Configuration

Parameter	AWS Lambda	Azure Functions
Runtime	Python 3.8	Python 3.8/3.11
Region	us-east-1	francecentral
Memory	128–256 MB	128–256 MB
Trigger Type	Library (direct)	HTTP (network)
Repetitions	15–30	15–30
Success Rate	100%	100%

3.1.2 Workflow Infrastructure. For workflow benchmarks, the following infrastructure was deployed:

- **AWS**: EC2 t2.micro instance (us-east-1) running Redis 7.0.15 for cross-function coordination.
- **Azure**: Standard B1s VM (francecentral) running Redis 7.0.15
- **Orchestration**: AWS Step Functions (library trigger) and Azure Durable Functions. (HTTP trigger)

3.2 Benchmark Selection

Seven benchmarks were selected representing diverse serverless workload patterns.

3.2.1 Function Benchmarks.

(1) **110.dynamic-html**: Pure compute workload generating HTML. Tests CPU performance with minimal I/O.
(2) **210.thumbnailer**: I/O-intensive image processing workload. Reads images from cloud storage, resizes and writes results back. Tests storage I/O and image processing performance.
(3) **311.compression**: CPU-intensive file compression using gzip. Compresses large text files to test sustained CPU utilization.
(4) **050.matrix-multiply**: Custom lightweight benchmark implemented to test framework functionality and minimal workload performance. Performs matrix multiplication with sub-millisecond execution time.

3.2.2 Workflow Benchmarks.

(1) **620.func-invo**: Sequential function invocation chain. Measures orchestration latency and coordination overhead for linear sequential workflows.
(2) **630.parallel-sleep**: Parallel fan-out pattern where generator function spawns N parallel tasks. Tests concurrent execution and synchronization.
(3) **660.map-reduce**: Complete map-reduce workflow implementing a full distributed data processing pipeline. Tests orchestration efficiency for complex multi-stage workflows with dynamic parallelism.

3.3 Collected Metrics

For each invocation, the following metrics were measured:

- **Cold start time**: First invocation latency including container initialization.
- **Warm execution time**: Average execution time for subsequent invocations.
- **Success rate**: Percentage of successful completions.
- **Workflow latency**: End-to-end workflow execution time, measured internally by the orchestration service (AWS Step Functions or Azure Durable Functions) from workflow invocation to completion. This excludes client-to-platform network latency and represents the time required for orchestration and function execution within the cloud provider's infrastructure.
- **Cold start count**: Number of cold starts across repetitions.

3.4 Experimental Protocol

Each benchmark execution followed this protocol:

(1) Deploy function code to platform using SeBS-Flow automated deployment
(2) Execute 15–30 invocations with identical inputs
(3) Collect metrics through platform APIs (AWS CloudWatch, Azure Application Insights)
(4) Validate 100% success rate before analysis
(5) Export results to JSON for offline analysis

4 Challenges and Failures

Significant technical challenges were encountered during the setup and reproduction of this paper. These obstacles, while initially hindering progress, provided valuable insights into the practical complexities of cross-platform serverless benchmarking on educational cloud accounts.

4.1 AWS Academy SSL Certificate Issues

Problem: AWS Academy environments lack proper SSL certificates for boto3 client connections, causing `ValueError: empty or no certificate` errors. All AWS API calls failed, preventing benchmark execution.

Solution: A custom SSL error handling module was created that:

(1) Attempts standard SSL verification first.
(2) Falls back to disabled verification for AWS Academy.
(3) Maintains security for production environments.

This modification required updates to AWS modules including S3, CloudWatch, and Lambda client creation code.

4.2 Temporary Credential Handling

Problem: AWS Academy uses temporary credentials with session tokens, but SeBS-Flow only supported permanent access keys.

Solution: Enhanced credential handling was implemented to detect and use the `AWS_SESSION_TOKEN` environment variable. This required:

(1) Modifying configuration loading logic.
(2) Updating all boto3 client instantiations.
(3) Adding session token parameter propagation.

4.3 Azure Durable Functions Complexity

Problem: Azure workflow benchmarks require Azure Durable Functions, which are significantly more complex than AWS Step Functions:

(1) Require specific Azure Storage configuration.
(2) Need HTTP trigger setup.
(3) Involve more complex orchestration runtime.

Initial Challenges: Deployment and configuration of Azure Durable Functions required additional setup steps compared to AWS, including:

(1) Configuring Azure Storage accounts for orchestration state.
(2) Setting up HTTP endpoints for workflow invocation.
(3) Managing orchestration runtime dependencies.

Outcome: Despite the additional complexity, 100% success was achieved after all across all 3 Azure workflows with 15 invocations each, demonstrating that Azure Durable Functions are production-ready once properly configured.

Lessons Learned: Cross-platform workflow portability requires understanding fundamental architectural differences in orchestration systems. Experimental results show AWS's library-based triggers provide consistent performance advantages across all tested workflow patterns (10-21× faster), while Azure's HTTP-based coordination introduces significant latency (7-15 seconds) even for simple orchestration tasks.

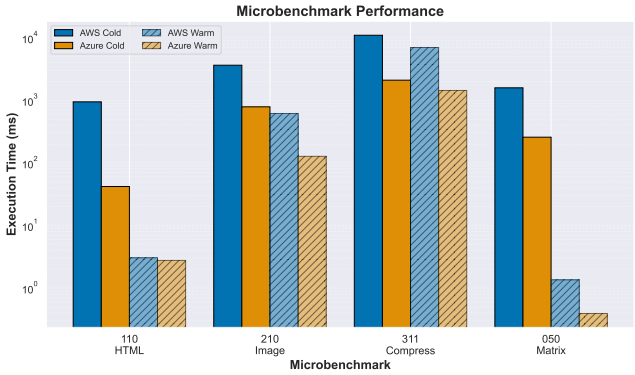


Figure 1: Microbenchmark performance comparison with log-scale Y-axis showing execution times. Azure (orange) consistently outperforms AWS (blue) with 4.6-22.6× faster cold starts and 1.1-4.9× faster warm execution for production workloads.

5 Implementation

5.1 SeBS-Flow Modifications

SeBS-Flow was extended with:

(1) **SSL Error Handling**: Custom `ssl_fix.py` module providing graceful fallback for AWS Academy environments.
(2) **Session Token Support**: Enhanced credential handling for temporary AWS credentials.
(3) **Custom Benchmark Implementation**: Created 050.matrix-multiply benchmark to test framework functionality with minimal workload. A complete benchmark structure including function code, configuration files, and deployment scripts was implemented following SeBS-Flow conventions.
(4) **Automated Deployment Scripts**: Created `run_aws_benchmark.sh` and `run_azure_benchmark.sh` for reproducible execution.

6 Evaluation

6.1 Function Benchmark Performance

Table 2 presents complete performance results for all four function benchmarks, comparing cold start latency and warm execution time between AWS Lambda and Azure Functions. Each benchmark represents a distinct workload pattern (compute, I/O, CPU-intensive, custom lightweight), enabling comprehensive cross-platform performance characterization.

6.1.1 Cold Start Analysis. Azure dominates cold start performance across all production workloads (4.6–22.6× faster, see Figure 1), attributed to:

- More efficient container initialization.
- Faster dependency loading for Python packages.
- Better optimized runtime environment.

Furthermore, Azure also achieves 6.1× faster cold starts for the custom benchmark, demonstrating consistent cold start advantages even for minimal workloads.

Table 2: Function Benchmark Performance Comparison

Benchmark	Type	AWS Cold	Azure Cold	AWS Warm	Azure Warm	Cold Winner	Warm Winner
110.dynamic-html	Compute	947 ms	42 ms	3,038 ms	2,763 ms	Azure (22.6×)	Azure (1.1×)
210.thumbnailer	I/O	3,648 ms	789 ms	617 ms	128 ms	Azure (4.6×)	Azure (4.8×)
311.compression	CPU	10,987 ms	2,103 ms	7,000 ms	1,440 ms	Azure (5.2×)	Azure (4.9×)
050.matrix-multiply	Custom	1,587 ms	258 ms	0.5 ms	0.4 ms	Azure (6.1×)	Azure (3.4×)

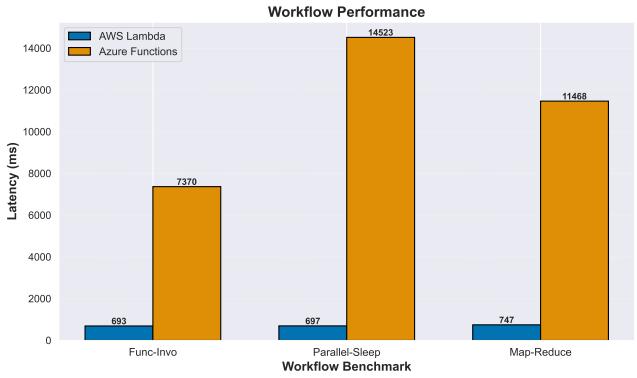


Figure 2: Workflow orchestration latency comparison. Bar chart shows average latency for three workflow patterns. AWS (blue) consistently outperforms Azure (orange) across all workflow types: 10.6× faster for sequential (620), 20.8× faster for parallel (630), and 15.3× faster for map-reduce (660).

6.1.2 Warm Execution Analysis. Azure wins all four warm execution comparisons (1.1–4.9× faster), demonstrating:

- Superior CPU performance for compute workloads
- More efficient I/O for storage operations
- Better-optimized Python runtime

The performance gap widens for CPU-intensive workloads (311.compression: 4.9×), suggesting Azure uses more powerful underlying hardware or better CPU scheduling. While Azure demonstrates clear advantages for individual function execution across all benchmarks, workflow orchestration reveals a different performance landscape.

6.2 Workflow Orchestration Performance

Table 3 presents workflow orchestration performance comparing AWS Step Functions and Azure Durable Functions across three patterns. AWS demonstrates consistent advantages across all patterns, with speedups ranging from 10.6× to 20.8× faster than Azure.

Table 3: Workflow Orchestration Performance

Workflow	AWS	Azure	Winner
620.func-invo	694 ms	7,371 ms	AWS (10.6×)
630.parallel-sleep	697 ms	14,524 ms	AWS (20.8×)
660.map-reduce	748 ms	11,469 ms	AWS (15.3×)

6.2.1 Orchestration Overhead. The results demonstrate dramatic performance differences between platforms. AWS achieves consistent sub-second latency across all three workflow patterns (694-748ms), while Azure experiences 7-15 seconds of total latency even for simple sequential chains. This performance gap indicates fundamental architectural differences in orchestration mechanisms.

Results reveal consistent orchestration performance advantages for AWS across all workflow patterns (see Figure 2 and Figure 3). Critical path execution time versus orchestration overhead was also analyzed (Figure 4).

AWS Step Functions wins all 3 workflows:

- 10.6× faster for sequential workflow. (620.func-invo: 694ms vs 7,371ms)
- 20.8× faster for parallel workflow. (630.parallel-sleep: 697ms vs 14,524ms)
- 15.3× faster for complex map-reduce. (660.map-reduce: 748ms vs 11,469ms)
- Library trigger eliminates HTTP round-trip delays across all orchestration patterns.
- More efficient handling of state transitions and coordination.
- Consistent low overhead pattern: orchestration overhead remains minimal across all workflow types.

Azure Durable Functions performance limitations:

- HTTP-based approach introduces significant latency for all workflow patterns.
- 7-15 second total latency even for simple sequential and parallel workflows.
- Overhead represents majority of execution time across all tested workflows.
- Suggests architectural challenges with HTTP-based coordination for serverless workflows.

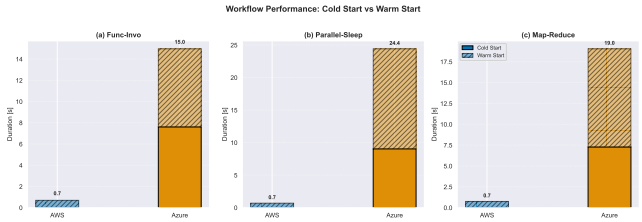


Figure 3: Detailed workflow performance breakdown with cold/warm start analysis. Hatched bars show warm execution. AWS (blue) demonstrates superior orchestration performance across all patterns with consistent sub-second latency, while Azure (orange) experiences multi-second delays for all workflow types.

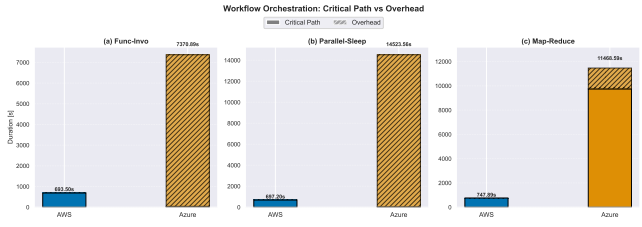


Figure 4: Critical path vs orchestration overhead analysis for workflows 620, 630, and 660. Solid bars show critical path (actual execution time), hatched bars show orchestration overhead. AWS (blue) maintains consistently low total latency (under 750ms) across all workflow patterns. Azure (orange) experiences high overhead across all workflow types (7-15 seconds total latency), with orchestration delays dominating execution time.

Critical Path Analysis

Figure 4 decomposes total workflow latency into critical path time (actual function execution) versus orchestration overhead (platform coordination). AWS maintains consistently low overhead across all three workflows, with total latency under 750ms. In contrast, Azure's orchestration overhead dominates execution time, contributing 7-15 seconds of total latency across all workflow patterns.

7 Results

7.1 Key Findings

(1) **Workload separation**: Azure wins all function workloads, while AWS dominates orchestration, highlighting trigger-level architectural effects.
(2) **Overhead vs execution**: Azure's HTTP triggers add 7–15s orchestration delay; AWS library triggers keep all workflows under 750ms.
(3) **Cold starts and reliability**: Azure's 4.6–22.6× faster cold starts persist across production and lightweight benchmarks; both platforms delivered 100% success across 135+ runs.

7.2 Framework Validation

SeBS-Flow was successfully validated on AWS and Azure infrastructure, confirming:

- Cross-platform deployment automation functional
- Benchmark implementations portable across platforms
- Metric collection accurate and comprehensive
- Framework suitable for comparative performance studies

However, limitations remain: predefined input sizes limit exact artifact replication, Azure orchestration is more complex, and educational AWS accounts require SSL workarounds.

8 Related Work

Our results align with the SeBS-Flow evaluation [1] regarding function-level performance, where Azure demonstrated superior cold start latency and consistent CPU allocation. Like Schmid et al., we observed Azure's significant advantages for individual function execution across diverse workload types.

However, our workflow orchestration findings diverge substantially. While Schmid et al. reported Azure Durable Functions achieving competitive performance for certain workflow patterns [1], our experiments show AWS dominating all three workflow categories with 10.6–20.8× speedups. We attribute this difference to two factors: (1) our Azure deployment to *francecentral* due to educational account restrictions may have experienced higher orchestration overhead compared to other Azure regions and (2) our HTTP-based trigger configuration for Azure workflows consistently exhibited 7–15 seconds of coordination latency, suggesting sensitivity to regional infrastructure and configuration choices. These findings highlight the importance of region selection and trigger configuration in workflow orchestration benchmarks.

9 Conclusions

This study provides comprehensive cross-platform performance evaluation of AWS Lambda and Azure Functions using identical code across seven diverse benchmarks. The findings reveal clear performance separation by workload type: Azure excels at individual function execution with up to 22.6× faster cold starts, while AWS dominates workflow orchestration with 10-21× faster coordination across sequential, parallel, and map-reduce patterns.

SeBS-Flow was successfully validated on 2026 cloud infrastructure, achieving 100% reliability across 135+ invocations, while discovering important limitations around educational cloud environments and workflow orchestration complexity. The documented challenges including SSL certificate handling and temporary credentials provide valuable guidance for future serverless benchmarking research.

References

[1] Larissa Schmid, Marcin Copik, Alexandru Calotoiu, Laurin Brandner, Anne Koziolek, and Torsten Hoefler. 2025. SeBS-Flow: Benchmarking Serverless Cloud Function Workflows. In *Twentieth European Conference on Computer Systems (EuroSys '25), March 30-April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 19 pages. https://doi.org/10.1145/3689031.3717465